Adapters

Implement the four adapters that talk to the outside world or to SQLite. Each hides its transport details behind a small interface.

4 tasks | Requires Wave 02 | Unlocks Wave 04

Why this wave exists

Wave 4 composes these adapters into the pipeline. Keeping the interfaces small is what makes the pipeline testable without WireMock.

How it fits

The diagram shows this wave's tasks, the earlier-wave tasks they depend on, and the later-wave tasks that depend on them.



Hand-off. Wave 4 composes product-context-client, availability-client, version-store and outbox-publisher into one pipeline.

Tasks

Implementing Product Context client

`product-context-client`

Depends on: retry-policy

Implement the client for GET /v1/products/{productId}/sizes/{sizeId}?countryCode=PT using the retry helper. Map 429 and 503 to retryable errors and 404 to a terminal NotFound. Return productStatus and countryEnabled so the pipeline can reject inactive products and disabled countries. Validate the response shape before returning it.

`src/clients/productContextClient.ts`

- [ ] style-90001 (429 with Retry-After: 5) and style-90002 (503) are handled as retryable.
- [ ] style-9999 (inactive) and style-9998 (countryEnabled false) map to domain rejections, not transport errors.
- [ ] Unknown combinations (404) are terminal.

Implementing version compare-and-set

`version-store`

Depends on: sqlite-schema

Implement the version compare-and-set on stock_versions. tryApply(key, sourceVersion) runs in one SQL transaction: insert the key if it is missing, then UPDATE ... WHERE applied_source_version < ?. One changed row means APPLIED; zero means STALE. Equal or lower versions never overwrite stock. Also expose read(key) for the re-check under the lease.

`src/stores/versionStore.ts`

- [ ] A higher version is applied; an equal or lower version returns STALE.
- [ ] Two calls with the same new version result in exactly one APPLIED.

Implementing outbox publisher

`outbox-publisher`

Depends on: retry-policy, sqlite-schema

Provide enqueue(tx, event), which writes an outbox row inside the caller's transaction, and a background publisher that reads unpublished rows and POSTs them to /mock-broker/v1/events. Publishing is parallel, bounded by CONCURRENCY. Each event carries sourceVersion, so consumers ignore any event whose version is not higher than the last one they applied. A 202 response marks the row as published. Failures are retried with backoff and recorded in attempts and last_error. Event ids are deterministic (stock key plus sourceVersion), so replays are safe.

`src/outbox/outboxPublisher.ts`

- [ ] published_at is set only after a 202 response.
- [ ] fault-429 and fault-503 event ids trigger retry behaviour at the sink.
- [ ] After a restart the publisher resumes unpublished rows.
- [ ] The consumer contract documents that consumers order by sourceVersion.

Implementing Availability API client

`availability-client`

Depends on: retry-policy

Implement PUT /v1/availability/{sellerId}/{productId}/{sizeId}?countryCode=PT with body availableQuantity, sourceVersion and the optional product context. The PUT is absolute, so retries are idempotent. Retry on 429 (honouring Retry-After), 503 and network errors. Map terminal 4xx responses to a rejection. Cover the seller-429 and seller-503 fault triggers.

`src/clients/availabilityClient.ts`

- [ ] seller-503 is retried and ends in success or a retryable-exhausted failure.
- [ ] The request body matches the contract in the assignment.

Risks and decisions

- The availability PUT is absolute. Never send a delta, or retries stop being idempotent.
- Parallel outbox publishing means consumers must order by sourceVersion. Document this in the consumer contract.

Verification

```
npm run typecheck
npm test -- clients version outbox
```

Done when

1. Every acceptance box is checked and the verification commands pass.
2. The session todos for this wave are marked done.
3. The final message includes a hand-off note for the next wave.

Agent brief for this wave: AGENT.md in the same folder.